

NORWEGIAN UNIVERSITY OF SCIENCE AND TECHNOLOGY
Department of Marine Technology

TMR4240 Marine Control Systems I

Project Part 1
Design of a Dynamic Positioning System

Python-Based Simulation and Control Design Guide

Contents

1	General Information	2
2	System Overview	5
3	Coordinate Frames, Notation, and Modelling Assumptions	6
3.1	Coordinate Frames	6
3.2	Six-DOF Notation and Three-DOF Reduction	7
3.3	Kinematic Equation	7
3.4	Process Plant Model (PPM)	8
3.5	Process Plant Model: R/V Gunnerus	9
3.6	Thruster Dynamics	9
3.7	Control Plant Model	10
4	Environmental Loads	10
4.1	Current Model	10
4.2	Wind Model	11
5	DP System Architecture	12
6	Controller Design	12
6.1	Possible Control Approaches	12
6.2	Tuning Methods	13
6.3	Anti-Windup	13
7	Reference Model	13
8	Thrust Allocation	14
8.1	Thruster Configuration	14
8.2	Configuration Matrix	14
8.3	Possible Allocation Variables	15
8.4	Allocation Methods	15
9	Mandatory Tests and Report	16
	Report Requirements	17
	Tips	18
A	Wind Coefficients	19

1 General Information

The report for Project Part 1 must be submitted in the Project module on Canvas by the deadline stated in the Lecture Plan. The submission must include the project report and either a ZIP archive or a repository link containing the executable Python code used for all simulations performed by the group. The appendix must briefly describe how the simulations can be executed and how the reported results can be reproduced.

Project Part 1 will be evaluated using the same general principles as Project Part 2. The assessment therefore considers not only whether the implementation works, but also the quality of the technical work, the clarity and correctness of the explanations, the quality of the presented results, and the ability to justify engineering decisions. Particular attention should be given to how simulation results are plotted, interpreted, and discussed. Only one report should be submitted per group. The recommended length is approximately 10–15 pages, excluding appendices, references, individual contribution statements, and reproducibility instructions, using the IEEE two-column format.

Project Objective

The main objective of Project Part 1 is to become familiar with the Python simulation framework for marine control systems and to design, implement, and validate a basic dynamic positioning (DP) system.

The project should be approached as an engineering development and validation process rather than simply as an implementation task. Students are expected to develop an understanding of how the different components of a DP system interact and to be able to explain and justify the modelling, control, simulation, and tuning choices made throughout the project.

A complete DP simulation involves several interconnected components. Consequently, the development should follow a structured process in which individual components are implemented and tested before the complete system is integrated.

Recommended Engineering Methodology

The project should generally follow the development process

Model $\rightarrow$ *Implement* $\rightarrow$ *Test* $\rightarrow$ *Tune* $\rightarrow$ *Validate* $\rightarrow$ *Integrate* $\rightarrow$ *Analyse*

Each subsystem should first be considered independently before being integrated into the complete DP system. Students should:

1. understand the purpose and assumptions of each subsystem;
2. implement the required functionality;
3. perform appropriate tests to verify that the implementation behaves as expected;
4. tune relevant parameters and explain the basis for the selected values;
5. validate the subsystem using suitable simulation cases;
6. integrate the validated subsystems into the complete DP system;

7. evaluate the behaviour of the complete system under relevant operating conditions; and
8. analyse and discuss the obtained results, including limitations and possible improvements.

The report should demonstrate this development process rather than only presenting the final implementation. Important modelling assumptions, design decisions, parameter choices, tuning procedures, and validation tests should be clearly documented and technically justified.

Simulation Results and Technical Discussion

Simulation results must be presented in a clear and technically meaningful manner. Figures should be appropriately labelled, contain relevant units and legends, and be sufficiently detailed to support the discussion.

Results should not be presented without interpretation. For each important simulation, the report should explain what was tested, why the test was performed, what behaviour was expected, and what can be concluded from the obtained results. Where appropriate, quantitative performance measures should be used to support the conclusions.

The discussion should also identify limitations and conditions under which the implemented system may not perform as intended. A successful result should therefore not only be shown, but also explained in terms of the underlying system behaviour.

Reproducibility

All simulations used to produce results presented in the report must be reproducible from the submitted code. The code should therefore be organized such that another student or member of the teaching staff can understand how the simulations are configured and executed.

The appendix should provide the necessary instructions for running the simulations and reproducing the main results. Any required dependencies, configuration steps, or relevant assumptions should be stated clearly.

Teamwork and Individual Responsibility

This is a teamwork project, but each student is individually responsible for contributing actively and professionally to the project. By the third week of the course, each group must agree on a clear division of responsibilities and communicate this division in a common email to the teaching staff.

Each student is expected to take ownership of their assigned tasks, maintain communication with the group, and seek assistance when difficulties arise. The division of responsibilities should support parallel development where appropriate, while ensuring that all group members understand how their work contributes to the complete DP system.

Although responsibilities may be divided between group members, the final system is a common group deliverable. Students should therefore communicate interface require-

ments, implementation changes, and relevant technical findings to the rest of the group throughout the development process.

Handling Difficulties and Incomplete Results

Unforeseen problems, unsuccessful approaches, and incomplete implementations are a normal part of engineering development. Such issues should be identified, documented, and communicated at an early stage.

If a component or part of the project cannot be completed as intended, this should be clearly described in the report. The report should explain what was attempted, what difficulties were encountered, when the problem was identified, and what was learned from the attempt. Where possible, students should also discuss potential causes of the problem and suggest how the implementation or methodology could be improved.

A technically well-documented unsuccessful attempt, supported by appropriate analysis and reflection, is preferable to an unexplained result or an implementation whose behaviour cannot be justified.

Individual Contributions and Assessment

The final report must include an individual contribution statement describing the responsibilities and contributions of each group member. The assessment may take individual contributions into account.

Individual assessment will consider not only the final group result, but also evidence of active participation, technical understanding, and professional engagement with the project. Students are expected to contribute consistently, communicate relevant difficulties in a timely manner, and take responsibility for their assigned tasks.

The overall objective is therefore not simply to obtain a working simulation, but to demonstrate an engineering approach in which the system is developed systematically, tested and validated, and supported by clear technical reasoning and reproducible results.

Final Note

This project is an opportunity not only to apply the concepts introduced in the lectures, but also to develop the engineering skills needed to approach complex problems systematically. You will likely encounter challenges along the way, but working through these challenges is an important part of the learning process.

Take the time to understand what you are doing, ask questions when something is unclear, learn from unsuccessful attempts, and support each other throughout the project. By the end of the course, the goal is not only to have a working DP system, but to have gained a deeper understanding of marine control systems and greater confidence in solving engineering problems independently.

Good luck, work well together, and enjoy the learning process!

— The teaching staff

2 System Overview

This project is the Python transition of the original TMR4240 Project Part 1, which was written for MATLAB/Simulink; the MATLAB version is not provided or used in this course edition. The same DP design problem is solved using the supplied Python template and the mcsimpy Gunnerus three-DOF vessel wrapper.

The link to the *GitHub template project repository* can be found in *Canvas* under the Project Module. Follow the *Quick Start* instructions in the repository *README.md*. The repository contains templates for both Part 1 and Part 2. Do not be afraid to modify the components provided in the template; the code is intended to be extended by the students.

The overall system architecture is shown in Fig. 1. The simulation consists of Process Plant Model (PPM), including environmental disturbances, and actuator dynamics, as well as the control modules. The student implementation comprises the following five subsystems:

- current loads model,
- wind loads model,
- DP controller,
- DP reference model,
- thrust allocation.

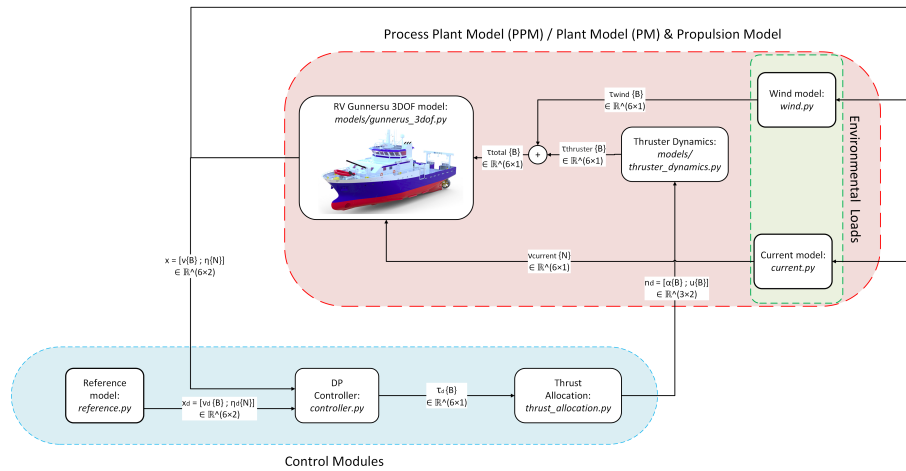


Figure 1: System overview of the DP-controlled Gunnerus vessel, showing the control modules, thruster dynamics, vessel dynamics, environmental disturbances, and feedback of the vessel state between the NED frame $\{N\}$ and body-fixed frame $\{B\}$.

Students are free to use any reasonable methodology for each subsystem provided that the choices are justified and documented in the report.

The supplied simulator engine handles the simulation loop, vessel state integration, actuator dynamics (saturation and rate limits that can be turned on and off), and logging. The student group implements the five subsystems listed above and produces a report that explains and validates the complete design.

3 Coordinate Frames, Notation, and Modelling Assumptions

Use this section of your report to define notation and assumptions before presenting any design. A reader should be able to understand the complete report without opening any simulation files. All symbol definitions must be consistent across sections.

3.1 Coordinate Frames

Two coordinate frames are used to describe the vessel motion: an Earth-fixed navigation frame and a body-fixed vessel frame.

The **Earth-fixed NED frame** is denoted by $\{n\}$ and is fixed to the Earth. Its axes are defined as x_n pointing North, y_n pointing East, and z_n pointing Down. This frame is used to describe the vessel position and heading relative to the surrounding environment.

The **body-fixed frame** is denoted by $\{b\}$ and is attached to the vessel. Its origin is typically chosen at the vessel centre of gravity or at another clearly specified reference point. The x_b axis points forward toward the bow and corresponds to surge motion, the y_b axis points to starboard and corresponds to sway motion, and the z_b axis points downward.

Both coordinate frames are right-handed. With the NED convention, a positive rotation about the downward z axis is clockwise when the vessel is viewed from above. Consequently, the yaw angle ψ is measured clockwise from North to the vessel bow direction.

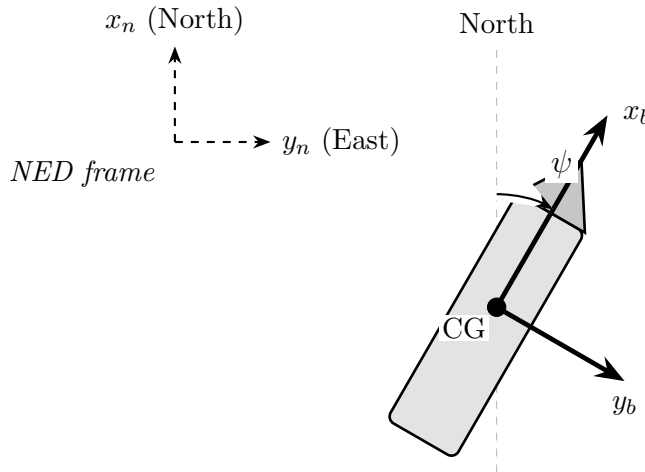


Figure 2: NED (Earth-fixed) and body (vessel-fixed) coordinate frames viewed from above. The NED frame axes x_n (North) and y_n (East) are shown as dashed arrows. The body-frame axes x_b (bow, surge) and y_b (starboard, sway) are solid. Both z axes point downward, into the page. Heading ψ is the clockwise angle from North to x_b .

Define every symbol before using it in equations or code. The report must introduce each vector and state its frame of reference.

3.2 Six-DOF Notation and Three-DOF Reduction

The standard six-DOF marine craft state vectors are:

$$\eta_6 = [x, y, z, \phi, \theta, \psi]^\top, \quad \nu_6 = [u, v, w, p, q, r]^\top, \quad \tau_6 = [X, Y, Z, K, M, N]^\top.$$

Report Task 1. Include a notation table in your report defining these vectors. Explain every component and state after each vector whether it is expressed in the NED frame or the body frame.

This project uses a horizontal-plane three-DOF model. Heave, roll, and pitch are neglected; only surge, sway, and yaw are retained:

$$\eta = [N, E, \psi]^\top, \quad \nu = [u, v, r]^\top, \quad \tau = [X, Y, N_z]^\top.$$

Report Task 2. Write the three reduced vectors explicitly and state which frame each is expressed in. This frame distinction is essential: vessel dynamics compute accelerations in body axes, while position is updated in NED.

3.3 Kinematic Equation

The kinematic equation converts body-frame velocity to NED position and heading rates. It is a geometric transformation only, not a force model. For three-DOF it reduces to a yaw-only rotation:

$$\dot{\eta} = J(\psi)\nu, \quad J(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Scalar form: $\dot{N} = u \cos \psi - v \sin \psi$, $\dot{E} = u \sin \psi + v \cos \psi$, $\dot{\psi} = r$.

For the three-DOF yaw rotation shown above, the inverse $J^\top(\psi) = J^{-1}(\psi)$ converts NED vectors to the body frame (e.g. a NED position error becomes a body-frame force direction). This transpose identity does not apply to the complete six-DOF Jacobian in general.

Heading errors must always be wrapped to $(-\pi, \pi]$:

$$\text{wrap}(\alpha) = \text{atan2}(\sin \alpha, \cos \alpha).$$

Report Task 3. Starting from the standard roll–pitch–yaw (z – y – x) convention, describe the role of the full six-DOF Jacobian $J(\eta_6)$ and then derive the 3×3 yaw-only case above. It is not necessary to reproduce every entry of the full 6×6 matrix. Write the three scalar three-DOF equations and describe where $J^\top(\psi)$ is applied in your implementation.

The key structural point: the plant model gives $\dot{\nu}$ in body coordinates, while the kinematic equation gives $\dot{\eta}$ in NED coordinates. These two together form the state derivative needed for integration.

3.4 Process Plant Model (PPM)

Process Plant Model (PPM), in literature also known as simply Plant Model (PM), is a detailed simulation model of the real physical process. Its purpose is not primarily control design, but to reproduce the real system closely enough to test whether the control system works under realistic conditions. The general six-DOF marine craft dynamics are:

$$M\dot{\nu}_6 + C(\nu_6)\nu_6 + D(\nu_r)\nu_r + g(\eta_6) = \tau_6.$$

Report Task 4. Define all terms in the six-DOF plant model and briefly state their physical meaning.

Current is handled through relative velocity. The NED current vector is rotated to the body frame and subtracted from the vessel velocity:

$$\nu_c^n = [V_{c,N}, V_{c,E}, 0]^\top, \quad \nu_c^b = J^\top(\psi) \nu_c^n, \quad \nu_r = \nu - \nu_c^b.$$

Here ν_c^n is the reduced three-DOF mathematical vector. In the Python interface, the same current is returned as the six-component array $[V_{c,N}, V_{c,E}, 0, 0, 0, 0]^\top$; only its horizontal components are used by the vessel model.

Report Task 5. Define all current variables and their coordinate frames. State whether the current direction is defined as “from” or “towards”.

For numerical simulation, the plant equation must be rearranged to solve for the vessel acceleration. The simulator state is $x = [\eta^\top, \nu^\top]^\top$.

Report Task 6. Rearrange the plant equation for $\dot{\nu}$ and write the complete state derivative $\dot{x}$:

$$\dot{\nu} = M^{-1}(\dots), \quad \dot{x} = \begin{bmatrix} J(\psi)\nu \\ M^{-1}(\dots) \end{bmatrix}.$$

The following implementation areas are especially relevant when working through Tasks 4–6: the vessel model including relative velocity, damping, Coriolis terms, and integration; the simulator loop including the ordering of reference, control, allocation, actuator limits, environmental loads, and vessel update; and the utility functions for rotations and angle wrapping.

The integration formulas used by the base vessel class are:

Forward Euler: $x_{k+1} = x_k + h f(t_k, x_k)$.

Runge-Kutta 4 (RK4):

$$\begin{aligned} k_1 &= f(t_k, x_k), & k_2 &= f(t_k + \frac{h}{2}, x_k + \frac{h}{2}k_1), \\ k_3 &= f(t_k + \frac{h}{2}, x_k + \frac{h}{2}k_2), & k_4 &= f(t_k + h, x_k + hk_3), \\ x_{k+1} &= x_k + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

State the chosen time step and integration method in the simulation setup section of the report. Note that the supplied vessel model supports forward Euler only (multi-stage integrators are rejected by the vessel wrapper); the RK4 scheme is shown for completeness.

3.5 Process Plant Model: R/V Gunnerus

The process plant used in the project is realised as a three-DOF maneuvering model of NTNU's research vessel R/V Gunnerus, provided by `mcsimpy`. It is a zero-frequency model with constant hydrodynamic coefficients (rigid-body mass approximately 574 t), implementing

$$\dot{\eta} = J(\psi)\nu, \quad (M_{RB} + M_A)\dot{\nu} + C_{RB}(\nu)\nu + C_A(\nu_r)\nu_r + D(\nu_r)\nu_r = \tau_{thr} + \tau_{wind} + \tau_{wave} + M_A\dot{\nu}_c^b,$$

with nonlinear damping

$$D(\nu_r) = D_L + D_u|u_r| + D_v|v_r| + D_r|r|.$$

Current acts only through the relative velocity ν_r and the term $M_A\dot{\nu}_c^b$; it is not applied as a separate force. There is no restoring term $g(\eta)$ in the horizontal plane. The numerical coefficient matrices are loaded from the supplied Gunnerus data file.

Report Task 7. Relate this model to the general six-DOF equation from Task 4 and its three-DOF reduction: explain what each term represents physically, which damping contributions are linear and which velocity-dependent, and which of these effects are dropped in the control plant model of Section 3.7.

3.6 Thruster Dynamics

In Part 1 the thruster dynamics are *disabled*: the simulator does not apply thrust-rate limits, azimuth-rate limits, or automatic saturation. The thrust allocator must nevertheless respect the static maximum thrust $u_{\max}$ in Table 1. Dynamic actuator limits are introduced in Part 2. *Thruster dynamics are disabled by default in the Part 1 settings.*

In reality, thrust is produced by a rotating propeller and is usually modelled through the open-water characteristics

$$T = \rho D^4 K_T(J_a) n|n|, \quad Q = \rho D^5 K_Q(J_a) n|n|, \quad J_a = \frac{V_a}{nD},$$

where n is the shaft speed, D the propeller diameter, and the thrust and torque coefficients K_T , K_Q are measured curves in the advance ratio J_a . A low-level controller tracks the shaft speed through the motor and shaft dynamics, subject to losses such as ventilation and thruster–thruster interaction [2, 1].

None of this is modelled in our simulator (at least for this year). For DP design purposes it is good enough to map the allocated commands (u_i, α_i) directly back into a body wrench with the configuration matrix of Eq. (3):

$$\tau_{thr} = B(\alpha) u.$$

This shortcut is exactly the mapping your allocator inverts: with ideal actuators, a correct allocator gives $B(\alpha_d)u_d = \tau_c$ identically. Use this as a test of your allocation (pure surge, sway, and yaw requests) before any closed-loop simulation.

The thruster dynamics block already implements all limits in Table 1, switched on by a single configuration flag. With the block disabled, commands are applied exactly and are not clipped; respecting $u_{\max}$ is therefore the responsibility of the allocator. Enabling the complete dynamics already in Part 1 is optional but beneficial: it makes the design more robust and gives a head start on Part 2. If you do, state it in the report.

3.7 Control Plant Model

The Control Plant Model (CPM), also known as Control Design Model (CDM), is a simplified mathematical description containing only the main physical properties of the process or plant; it can form part of the controller and is used for analytical stability analysis. Write the control plant section of the report as follows:

- Control plant model equations.
- State the simplifying assumptions.
- Give the numerical matrices or state their source.
- Explain why this simplified model is adequate for controller design despite the non-linear effects in the simulation model.

If a state-space method (LQR, pole placement) is used, also write equations in state space form:

$$\dot{x}_c = A_c x_c + B_c \tau_c, \quad y_c = C_c x_c.$$

Define the state ordering and all matrices. If no state-space method is used, omit this form.

Note. The control plant is a design tool. A simple linear control plant is appropriate for DP even when the process plant includes nonlinear effects. The gap between process plant model and control plant model is the reason why tuning is always iterative in practice.

4 Environmental Loads

Explain how current and wind are represented in the simulation. State what is an input, what is computed, and what is neglected. Environmental loads in Part 1 are current and wind only. Waves are reserved for Part 2. Ice loads, mooring lines, and other external forces can be disregarded.

4.1 Current Model

The first part is the implementation of the current drag effects on the vessel. The current shall be defined in the NED frame, but only the North and East coordinates are needed (the current average down component will always be 0).

The effects of the current forces/moment don't need to be calculated, since the vessel model already accounts for the drag effects.

Describe current magnitude, direction, and the frame transformation from NED to the body frame. Identify the direction convention from your own implementation and state whether the input direction means “from” or “towards”. State also whether current enters the vessel model as a separate load or through relative velocity.

Hint. “Current from east” means the water flows westward, giving a negative East component in the NED current vector. A 180° sign error in the direction convention causes the controller to oppose the correct response.

4.2 Wind Model

The wind model must return a six-component generalized BODY-frame load vector to follow the common simulator interface. The three-DOF plant uses only indices $[0, 1, 5]$, corresponding to surge force, sway force, and yaw moment; the remaining entries do not affect the Part 1 vessel model. Describe the wind force model including wind speed, direction, relative wind angle, and coefficient interpolation. In Project Part 1 the wind speed shall have a mean component and a slowly varying component, while the wind direction does not vary.

The wind model implemented shall use a coefficient table, and the following equation:

$$\tau_{\text{wind}} = U_{rw}^2 C_w(\alpha_{rw}), \quad (1)$$

where U_{rw} is the *relative* wind speed and $C_w(\alpha_{rw}) \in \mathbb{R}^6$ is the tabulated and interpolated wind coefficient vector, $C_w = [C_x, C_y, C_z, C_\phi, C_\theta, C_\psi]^\top$. The entries of C_w are dimensional ($\text{N s}^2/\text{m}^2$ for the forces, $\text{N m s}^2/\text{m}^2$ for the moments): air density and windage areas are already absorbed into the table. Note that C_z and C_θ are zero in the provided table, and the 3-DOF simulator uses only the surge, sway, and yaw components (C_x, C_y, C_ψ); the full six-component vector is kept so that all load interfaces share the same format. The relative wind velocity is the ambient wind minus the vessel velocity, expressed in the body frame; it defines both the relative wind speed U_{rw} and the relative wind angle α_{rw} :

$$\begin{bmatrix} V_{rw,x}^b \\ V_{rw,y}^b \end{bmatrix} = J^\top(\psi) \begin{bmatrix} V_{w,N} \\ V_{w,E} \end{bmatrix} - \begin{bmatrix} u \\ v \end{bmatrix},$$

$$U_{rw} = \sqrt{(V_{rw,x}^b)^2 + (V_{rw,y}^b)^2}, \quad \alpha_{rw} = \text{atan2}(V_{rw,y}^b, V_{rw,x}^b).$$

At DP speeds the difference between ambient and relative wind speed is small, but the automated checks in the template verify the relative-wind implementation.

Wind coefficients are tabulated as a function of α_{rw} and must be interpolated; see Appendix A. In Fig. 5, α is the relative wind angle, whose geometry is defined in Fig. 3.

Recommendation. Verify the wind model with a simple test before connecting it to the DP loop: set heading $\psi = 0$ (bow north) and apply wind from the north (velocity points south), i.e. $\alpha_{rw} = 180^\circ$. The expected result is a negative surge force and near-zero sway force.

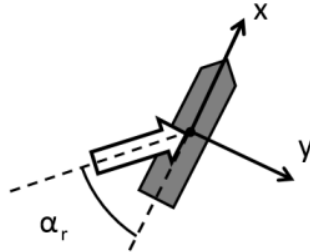


Figure 3: Definition of the relative wind angle.

5 DP System Architecture

The report must include an original architecture figure created by the student group. Do not only redraw standard textbook blocks: explain what each block does, what signal enters and leaves it, and which frame each signal belongs to. Part 1 has no observer; the simulated vessel states are fed back directly.

Warning. Frame conversions are critical at two points: (1) current in NED is rotated to the body frame before entering the vessel model; (2) NED position error in the controller is rotated to the body frame before becoming a force command. Both errors are common and difficult to detect without targeted tests.

6 Controller Design

The DP controller shall be responsible for the vessel dynamic positioning. It takes the vessel to the desired setpoint and then keeps it there, given the environmental loads (which are typically unknown to the DP). The chosen controller shall receive information about the vessel's actual position (NED) and the desired setpoint (NED), and based on that calculate the desired forces in surge and sway as well as yaw moment. All units shall follow the SI (N and Nm). In general, the DP controller drives position and heading error to zero while minimising control effort:

$$e_\eta = \eta_d - \eta, \quad e_\psi = \text{atan2}(\sin(\psi_d - \psi), \cos(\psi_d - \psi)).$$

The heading error must always be computed with `atan2` to enforce wrapping to $(-\pi, \pi]$. Good DP performance means small steady-state error, acceptable transient overshoot, smooth response, and reasonable actuator effort.

Explain design choices, not just final gains. Justify the chosen method and show how it was tuned. The report must explain: the control objective; error definitions; how NED position errors are converted to body-frame force commands; how heading error is wrapped; controller gains or weighting matrices; the tuning procedure; and how integral growth or infeasible force demands are handled.

6.1 Possible Control Approaches

PID Three PID channels for N , E , ψ . The integral term eliminates steady-state error under constant disturbances. The position channels require a frame rotation: NED error $\rightarrow J^\top(\psi) \rightarrow$ body-frame force command. An anti-windup scheme is recommended when saturation occurs.

PD with feedforward If the reference model outputs $\dot{\eta}_d$, the feedforward term improves tracking and reduces the required integral gain.

LQR State-feedback law $u = -Kx$ derived from the algebraic Riccati equation. Requires linearising the control plant. Offers systematic tuning through weight matrices Q (state penalty) and R (effort penalty).

Recommendation. For all methods: always state which signals are in NED and which are in the body frame. Many DP bugs arise from applying NED-frame position errors directly as body-frame force commands without the rotation $J^\top(\psi)$.

6.2 Tuning Methods

1. **Frequency-domain design.** Choose closed-loop bandwidth. For each translational axis:

$$K_p = m\omega_c^2, \quad K_d = 2\zeta_c\omega_cm,$$

where m is the added-mass-corrected inertia and ζ_c is the desired damping ratio.

2. **LQR design.** Pose $J = \int (x^\top Qx + u^\top Ru) dt$ and solve the algebraic Riccati equation; see [1], Sec. 13.2.
3. **Iterative simulation.** Start from a physically motivated guess. Run a step-response simulation, observe overshoot and settling time, and adjust gains. This is the most common approach in practice.

Recommendation. Document the tuning procedure: state the starting point, the observation driving each adjustment, and the final values. A table of gains with physical effect (position accuracy, damping, steady-state error) is required in the report. Discuss the trade-off between speed, overshoot, steady-state error, and control effort.

6.3 Anti-Windup

Integrator wind-up occurs when the requested wrench cannot be achieved and the integrator continues accumulating error, causing large overshoot when the system returns to a feasible operating region. This can occur when the allocator enforces the static thrust limits, or when the optional actuator dynamics are enabled. A back-calculation scheme prevents this:

$$\dot{x}_i = e + K_{aw}(\tau_{\text{sat}} - \tau_{\text{unsat}}),$$

where $K_{aw} > 0$, τ_{sat} is the achievable or applied command, and τ_{unsat} is the requested command.

7 Reference Model

The key idea is to avoid applying abrupt setpoint steps directly to the controller and thrusters. Without a reference model, a large setpoint jump instantly requests a large control output, potentially saturating the thrusters. The reference model generates a smooth desired trajectory $\eta_d(t)$ from the commanded step η_{sp} .

Describe how the reference model generates smooth desired position, velocity, and acceleration arrays from commanded setpoints, and explain why this helps the controller and reduces actuator effort. The Python interface always returns all three arrays; a controller may choose not to use velocity or acceleration feedforward. State the parameter values used in each simulation and justify them.

A second-order low-pass filter per controlled coordinate is the standard choice:

$$\ddot{\eta}_d + 2\zeta_r\omega_r\dot{\eta}_d + \omega_r^2\eta_d = \omega_r^2\eta_{sp}, \quad (2)$$

where ω_r is the natural frequency and ζ_r the damping ratio.

Recommendation. Critical damping ($\zeta_r = 1$) is almost always the right choice for DP. Set ω_r so that the time constant $T_r = 1/\omega_r$ is comfortable for the thrusters.

8 Thrust Allocation

Thrust Allocation is the interface between the controller, which works in generalised forces, and the propulsion system, which produces forces at specific locations and in specific directions.

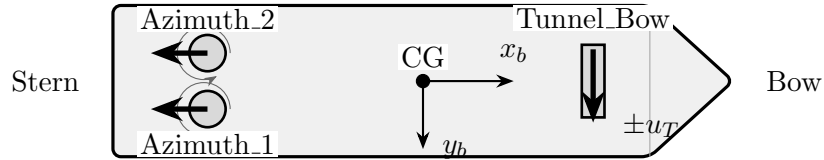
Do not only state that “the allocator computes thrusts.” Explain the mathematical problem, the decision variables, and how the solution converts to per-thruster commands. Keep in mind that this system is over-actuated, so it is possible to optimize the thrusters setpoint to minimize power consumption. Also, thruster angle is given in the body frame, clockwise, with zero pointing to the bow. The allocator must respect the static thrust magnitudes $u_{\max}$ in Table 1. Thrust-rate and azimuth-rate constraints may be disregarded in Part 1 because the actuator dynamics are disabled by default.

8.1 Thruster Configuration

The Python template uses the following three-thruster layout:

Table 1: Thruster configuration. Positions from CG in body frame ($x>0$ forward, $y>0$ starboard). Angle α_0 measured from x_b toward y_b (clockwise positive when viewed from above).

Thruster	Type	x [m]	y [m]	α_0	$u_{\max}$ [kN]	$\dot{u}_{\max}$ [kN/s]	$\dot{\alpha}_{\max}$ [°/s]
Tunnel_Bow	tunnel	+12.0	0.0	90° (fixed)	32	4	0
Azimuth_1	azimuth	-13.0	+3.0	0° (free)	80	10	12
Azimuth_2	azimuth	-13.0	-3.0	0° (free)	80	10	12



Azimuth thrusters can rotate ($\dot{\alpha}_{\max} = 12^\circ \text{ s}^{-1}$). Tunnel thruster has fixed direction.

Figure 4: Three-thruster layout (top view, body frame, bow to the right).

8.2 Configuration Matrix

For thruster i at position (x_i, y_i) with magnitude u_i and angle α_i , the produced wrench column is:

$$B_i(\alpha_i) = \begin{bmatrix} \cos \alpha_i \\ \sin \alpha_i \\ x_i \sin \alpha_i - y_i \cos \alpha_i \end{bmatrix}. \quad (3)$$

The total wrench is $\tau = B(\alpha)u$. Build the matrix column by column from Table 1. For the tunnel thruster ($\alpha_T = 90^\circ$):

$$B_{\text{Tunnel}} = \begin{bmatrix} 0 \\ 1 \\ 12 \end{bmatrix}.$$

Recommendation. Do not only write the final matrix. Explain one column in words (e.g. the fixed tunnel angle gives fixed surge/sway components; stern azimuth columns are angle-dependent). Verify $\text{rank}(B) = 3$ for the chosen angles. Also inspect columns physically: a thruster close to the CG gives little yaw authority; one far from the CG provides a larger moment for the same force.

8.3 Possible Allocation Variables

Directly solving for azimuth magnitude u_j and angle α_j gives a nonlinear problem. A common linearisation uses Cartesian force components:

$$F_{x,j} = u_j \cos \alpha_j, \quad F_{y,j} = u_j \sin \alpha_j.$$

A natural decision vector is $z = [u_T, F_{x,1}, F_{y,1}, F_{x,2}, F_{y,2}]^\top$, giving the extended matrix:

$$B_e = [B_T \quad B_{Fx,A1} \quad B_{Fy,A1} \quad B_{Fx,A2} \quad B_{Fy,A2}], \quad B_{Fx,j} = \begin{bmatrix} 1 \\ 0 \\ -y_j \end{bmatrix}, \quad B_{Fy,j} = \begin{bmatrix} 0 \\ 1 \\ x_j \end{bmatrix}.$$

Fill in numerical entries from Table 1 and verify $\text{rank}(B_e) = 3$.

Recommendation. After solving for Cartesian components, recover magnitude and angle: $u_j = \sqrt{F_{x,j}^2 + F_{y,j}^2}$, $\alpha_j = \text{atan2}(F_{y,j}, F_{x,j})$. Always use `atan2`, not `atan`.

8.4 Allocation Methods

Before choosing a method, answer:

- What are the decision variables? Is the problem linear or nonlinear?
- Are all wrench components penalised equally, or is yaw weighted differently?
- What happens if the requested wrench exceeds actuator capacity?
- How are unnecessary azimuth rotations avoided?

Common methods:

Pseudo-inverse $z = B_e^\top (B_e B_e^\top)^{-1} \tau_c$. Minimises $\|z\|^2$. Simple but does not enforce saturation.

Weighted least squares $z = \arg \min_z \frac{1}{2} z^\top W z$ s.t. $B_e z = \tau_c$. Weight matrix W penalises components differently.

Damped least squares $z = B_e^\top (B_e B_e^\top + \lambda I)^{-1} \tau_c$. Damping λ prevents large commands near singularity.

Implementation hints. Check $\text{rank}(B_e)$ before long simulations. Wrap recovered angles to $(-\pi, \pi]$. Use a linear solver rather than computing an explicit matrix inverse. Test the allocator on isolated wrenches (pure surge, pure sway, pure yaw) before connecting to the full DP loop.

Recommendation. Angle ambiguity is easy to miss: a positive thrust at α and a negative thrust at $\alpha + \pi$ produce the same force. Choose the representation requiring the smallest azimuth rotation from the current angle.

9 Mandatory Tests and Report

Several tests are required to show that your system is working properly. They are described in the following sections. All plots must have readable axis labels with units, legends where needed, and every figure must be discussed in the text.

Simulation 1: Station Keeping

This simulation consists of two different subsimulations that show the DP capabilities of your DP system with two different environmental conditions. DP setpoint $\eta_{sp} = [0, 0, 0]^\top$. Two separate cases:

- Current at 0.5 m/s from east. No wind.
- Wind with average speed 15 m/s from east, with a slowly varying component of standard deviation 1 m/s. No current.

Plot $N(t)$, $E(t)$, $\psi(t)$, and xy -trajectory for both cases until steady state.

Recommendation. Present the two cases separately. Discuss maximum position error, steady-state error, settling time, heading error, and actuator effort for each case.

Simulation 2: Rotating Current

Turn off wind. Make the current vary linearly from 0.5 m/s from north to 0.5 m/s from east over 300 s, keeping the vessel at the origin. The current *direction* varies linearly while the speed stays constant at 0.5 m/s. Run longer if needed to reach steady state. Plot position, heading, and the actual NED current components $V_{c,N}(t)$ and $V_{c,E}(t)$ returned to the plant. Note that a scenario described as “current from east” produces a westward velocity and therefore a negative East component.

Recommendation. Show the current input as part of the results (e.g. $V_{c,N}(t)$, $V_{c,E}(t)$, or direction versus time). Discuss how the controller reacts during the 300-second transition and after the current stabilises.

Simulation 3: Setpoint Change With and Without Reference Model

Compare the vessel response over time for initial position $\eta_0 = [0, 0, 0]^\top$ and setpoint $\eta_{sp} = [10, 10, 3\pi/2]^\top$. Note that $3\pi/2$ lies outside $(-\pi, \pi]$ and is physically equivalent to $-\pi/2$. The controller and reference model shall use the shortest angular displacement; from the initial heading, the vessel should therefore turn toward $-\pi/2$ rather than complete an unnecessary positive rotation. Compare with and without reference model, both without environmental forces. Plot position and heading until steady state as time series and xy -plots. Include the desired trajectory. If the reference model outputs velocities, plot them alongside actual values.

Recommendation. Use identical controller and allocation settings for both runs so that the reference model effect is isolated. Compare overshoot, oscillation, settling time, trajectory smoothness, and actuator effort. Explain *why* the reference model changes the response.

Simulation 4: Four-Corner Test

The last mandatory simulation is to perform a DP four-corner test without environmental forces. Pass through the following setpoints in order, keeping steady position at each before moving to the next:

$$\begin{aligned}\eta_0 &= [0, 0, 0]^\top, & \eta_1 &= [50, 0, 0]^\top, & \eta_2 &= [50, -50, 0]^\top, \\ \eta_3 &= [50, -50, -\pi/4]^\top, & \eta_4 &= [0, -50, -\pi/4]^\top, & \eta_5 &= [0, 0, 0]^\top.\end{aligned}$$

Use a fixed hold time of 300s per listed setpoint; this is the prescribed reproducible profile used by the automated checks. Verify from the results that the vessel has reached an acceptably steady position before each transition. If it has not, discuss the cause rather than silently changing the mandatory test profile.

Plot the position and heading until you reach steady state, both as individual and in an xy-plot. In addition, put the desired trajectory in the individual plots. If the reference model contains velocity trajectories, then plot these with the actual velocities.

Report Requirements

The following sections should be present in the Part 1 report:

- Introduction
- Coordinate frames
- Process plant model (including kinematic equation)
- Control plant model
- Current model
- Wind model
- DP system architecture (original figure required)
- Reference model
- Thrust allocation
- Controller design (method, tuning procedure, gains)
- Simulation results: Simulations 1–4 with all required plots
- Conclusion (Discussion)
- Individual contributions (factual and specific)
- Appendix: how to reproduce all simulations

Wave model, and observer design are Part 2 topics. Recommended length: approximately 10–15 pages, excluding appendices, references, individual contribution statements, and reproducibility instructions.

Submit the report together with the simulation files or a repository link.

Tips

- **Work block by block.** Do not try to solve the complete system at once. Test current, wind, controller, reference model, and thrust allocation separately before running the full closed-loop simulation.
- **Perform simple sign tests.** Check that position errors, environmental forces, and body-frame transformations produce forces in the expected directions for different vessel headings.
- **Check heading wrapping.** Test setpoints near π and $-\pi$. The controller should follow the shortest rotation and should not produce an unnecessary 2π turn.
- **Use consistent units.** Use metres, seconds, radians, newtons, and newton-metres throughout the implementation. In particular, avoid mixing degrees and radians.
- **Check the thrust-allocation matrix.** Verify that $\text{rank}(B_e) = 3$ before running long simulations, and check for static thrust-limit violations or infeasible force demands.
- **Evaluate the expected station-keeping performance.** For the environmental station-keeping tests, after the initial transient, a reasonably tuned solution should normally keep the position error within approximately 1 m–5 m. This guideline does not apply to the intentional setpoint displacements in Simulations 3 and 4. Significantly larger station-keeping errors may indicate frame or sign errors, poor controller tuning, insufficient allocation capability, or infeasible actuator demands. If the error is large, discuss the likely cause.
- **Present the results clearly.** Figures and simulation results should contain enough information for the reader to understand what was simulated, under which conditions, and how the system performed. Remember that the reader only knows what is presented in the report.
- **Treat the project as an engineering task.** A good implementation is important, but so are a clear system structure, readable plots, suitable interfaces, and a well-written report.
- **Plan your group work.** Even with four group members, the project can be surprisingly time-consuming alongside other course work. Start early and avoid leaving the main integration and testing to the last weekend.

A Wind Coefficients

Wind coefficients as a function of relative wind angle α_{rw} are provided in the project files as a data table *data/wind_coeff.csv* (Fig. 5), where α_{rw} is the direction the relative wind blows *towards* in the body frame (0° = following wind, 180° = head wind). The wind loads are $\tau_{\text{wind}} = U_{rw}^2 C(\alpha_{rw})$; air density and windage areas are already absorbed into the coefficients.

The table is generated for R/V *Gunnerus* with the method of Blendermann [7] (ship type *research vessel*). C_z and C_θ are zero and unused by the 3-DOF simulator. Generate the coefficient plot from `notebooks/wind_coefficients.ipynb` in the project template to verify interpolation quality before connecting the wind model to the DP loop.

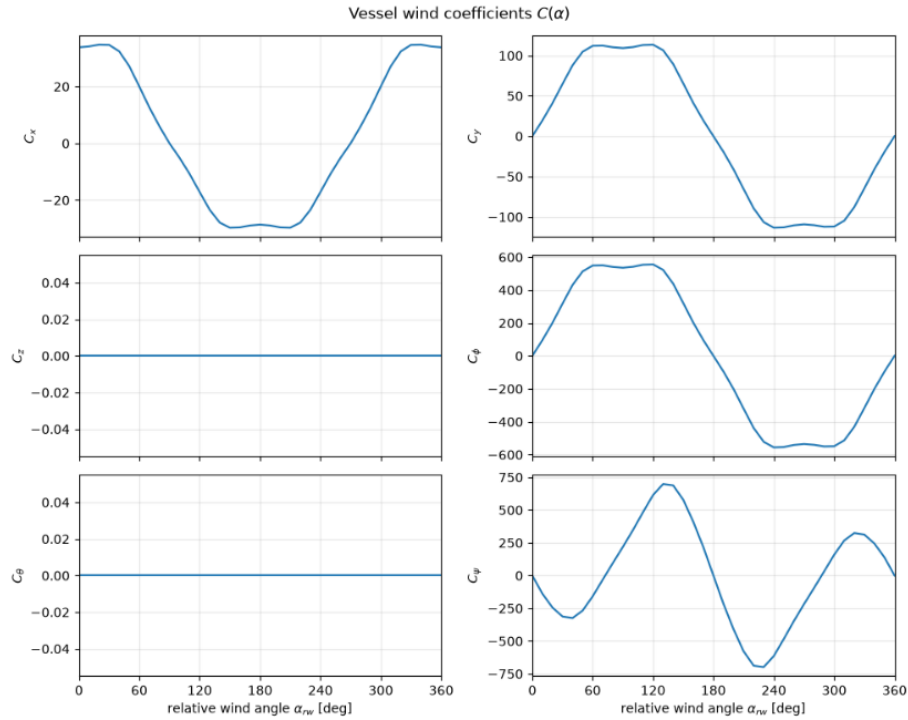


Figure 5: Gunnerus wind coefficients $C_w(\alpha_{rw})$, generated with the Blendermann method.

References

- [1] T. I. Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control*. Chichester, UK: Wiley, 2011. [*Primary reference. Key chapters: 2 (notation), 6–7 (vessel dynamics), 8 (env. loads), 12 (DP), 13 (control design).*]
- [2] A. J. Sørensen, *Marine Cybernetics: Towards Autonomous Marine Operations and Systems*. Lecture Notes, Dept. Marine Technology, NTNU, Report UK-18-76, 2018. [*TMR4240 lecture notes. Key chapters: 6 (modelling), 7 (DP design).*]
- [3] A. J. Sørensen, “A survey of dynamic positioning control systems,” *Annual Reviews in Control*, vol. 35, no. 1, pp. 123–136, 2011. [*Good overview of DP architecture, operational modes, and control approaches.*]
- [4] A. J. Sørensen, S. I. Sagatun, and T. I. Fossen, “Design of a dynamic positioning system using model-based control,” *Control Engineering Practice*, vol. 4, no. 3, pp. 359–368, 1996. [*Foundational paper on model-based DP design; describes the control plant approach used in this project.*]
- [5] J. G. Balchen, N. A. Jenssen, and S. Sælid, “Dynamic positioning using Kalman filtering and optimal control theory,” in *Proc. IFAC Conf. Automation in Offshore Oil Field Operation*, Bergen, Norway, 1980. [*The original DP control paper. Introduced Kalman filtering and LQG design to dynamic positioning.*]
- [6] O. J. Sjørdalen, “Optimal thrust allocation for marine vessels,” *Control Engineering Practice*, vol. 5, no. 9, pp. 1223–1231, 1997. [*Describes the weighted least-squares thrust allocation approach.*]
- [7] W. Blendermann, “Parameter identification of wind loads on ships,” *Journal of Wind Engineering and Industrial Aerodynamics*, vol. 51, no. 3, pp. 339–351, 1994. [*Empirical wind-load coefficients by ship type; used to generate the provided wind coefficient table.*]